

Wild Odds Gym Tracker

As-Built & Configured Reference

A complete snapshot of the application as it currently exists and is configured — architecture, data model, every feature, configuration, and integration points — intended as the grounding context for AI-assisted planning and a new-feature roadmap.

Android application • package **com.wildodds.gymtracker**
Version 2.0.0 (versionCode 2) • Database schema v13
Document generated 19 June 2026

Contents

1. How to use this document
2. Product overview
3. Technology stack & build configuration
4. Application architecture
5. Package / module map
6. Data model (Room schema v13)
7. Repository & DAO layer
8. Navigation map
9. Feature catalogue
10. Import & parsing subsystem
11. Default program catalogue (asset-driven)
12. Configuration, feature flags & secrets
13. Sharing & FileProvider
14. Background work, receivers & home-screen widget
15. Data & security model (access control / ABAC)
16. Build, signing & release artifacts
17. Quality: tests & static analysis
18. Known constraints, tech debt & risks
19. Extension points (where new features plug in)
20. Recent changes (this work cycle)
21. Appendix — source file index

1. How to use this document

This is an **as-built** reference: it describes what the codebase does *today*, not an aspirational design. It is written to be pasted (in whole or in part) into an AI assistant as grounding context when planning new features, so that the assistant reasons about the real architecture instead of guessing.

- **Sections 2–8** establish the shape of the system: stack, architecture, data model, and navigation.
- **Section 9** is the feature catalogue — the “what the app does” inventory, screen by screen.
- **Sections 10–14** cover the subsystems that are easy to overlook (import/parsing, configuration, sharing, background work).
- **Sections 15–18** are the honest engineering picture: security posture, build/release reality, test coverage, and debt.
- **Section 19** is the most useful for roadmapping: the concrete seams where new capability attaches.

Where a fact is a deliberate design choice or a known limitation, it is called out so the roadmap can treat it as a decision point rather than an accident.

2. Product overview

Wild Odds Gym Tracker is a **single-user, offline-first Android app** for planning and logging resistance-training programs. A user runs one *active program* at a time, works through its weeks and days, and logs weight / reps (plus optional RPE, %1RM, and per-side values) for each set. The app emphasises a frictionless logging loop (previous-week values carry forward as editable prefill) and a large built-in catalogue of ready-made programs.

Primary capabilities at a glance

- Run a program week-by-week / day-by-day with set logging and automatic carry-forward of last week's numbers.
- Import programs from Excel (smart auto-detection), a bundled 61-program catalogue, bulk multi-program/phased workbooks, or any file via AI.
- Build custom programs and reusable session templates in-app.
- Multi-phase periodised programs (e.g. Accumulation / Intensification / Peak) with a phase selector.
- Program analysis dashboard (weekly set volume per muscle group, push/pull balance, insights).
- History of completed sessions with training volume; a separate daily Habit tracker with a home-screen widget.
- Theming (dark mode, adjustable accent colour) and a set of toggable convenience features.

It is a **local app with no backend**. The only outbound network call is an optional, user-initiated request to the Anthropic Claude API for AI-assisted import (the user supplies their own API key).

3. Technology stack & build configuration

Area	Choice / value
Language	Kotlin 1.9.24 (JVM target 17, Java 17 source/target)
UI toolkit	Jetpack Compose (BOM 2024.06.00 = Compose UI 1.6.8), Material 3, Compose compiler ext 1.5.14
Architecture	MVVM + single Repository; Kotlin Coroutines / Flow; AndroidX Lifecycle 2.8.2
Persistence	Room 2.6.1 (KSP codegen); Jetpack DataStore (Preferences) 1.1.1; SharedPreferences
Navigation	Navigation-Compose 2.7.7 (single-Activity, Compose nav graph)
Async / net	kotlinx-coroutines 1.8.1; OkHttp 4.12.0 (Claude API); Gson 2.11.0 (JSON)
Images	Coil 2.6.0 (program cover images)

Area	Choice / value
Widget	Glance 1.1.0 (App Widget for habits)
Docs/files	PdfBox-Android 2.0.27.0 (used by the AI file-text extractor)
Build	AGP 8.4.2; KSP 1.9.24-1.0.20; Gradle KTS; version catalog (libs.versions.toml); multidex
SDK levels	compileSdk 36, targetSdk 36, minSdk 26 (Android 8.0+)
App identity	applicationId com.wildodds.gymtracker; versionName 2.0.0; versionCode 2

Note: AGP 8.4.2 is officially tested only to compileSdk 34, so the build emits a compileSdk-36 advisory warning (it compiles and packages fine). The project is a **single Gradle module** (:app).

4. Application architecture

The app follows a conventional **MVVM + Repository** layering. Compose screens observe `StateFlow` exposed by `AndroidViewModels`; view models call a single `GymRepository`; the repository is the only component that touches Room DAOs and orchestrates multi-table operations on the IO dispatcher.

```

UI (Jetpack Compose screens)
  observe StateFlow / collectAsStateWithLifecycle
  |
ViewModels (AndroidViewModel, one per screen area)
  HomeViewModel, SessionViewModel, SessionInfoViewModel, LibraryViewModel,
  SessionLibraryViewModel, CreateProgramViewModel, HistoryViewModel,
  HabitsViewModel, SettingsViewModel
  |
GymRepository (single data choke point; withContext(Dispatchers.IO))
  |
Room DAOs -> AppDatabase (SQLite 'gym_tracker.db', schema v13)

Cross-cutting: ThemePreferences (DataStore), SharedPreferences (gym_prefs,
exercise_notes, favourites), assets/ (xlsx catalogue + templates),
ClaudeApiClient (OkHttp -> Anthropic API)

```

- **Single Activity:** `MainActivity` (`ComponentActivity`) sets the Compose content, enables edge-to-edge, and wires theme state from `ThemePreferences`. `GymApp` is the `Application` class.
- **Reactive home:** the Home screen derives entirely from `Flows` (`currentProgram`, all sessions, all completions, phases), so any data mutation (e.g. completing or resetting a day) refreshes the UI automatically.
- **Repository owns transactions:** operations that span tables (import, reorder, propagate-to-future-weeks, reset, restart) live in `GymRepository` and run off the main thread.
- **No DI framework:** dependencies are constructed directly (`AppDatabase.getInstance()` singleton, repository instantiated per view model).

5. Package / module map

All code lives under `com.wildodds.gymtracker` in the single `:app` module.

Package	Responsibility
(root)	<code>MainActivity</code> , <code>GymApp</code> (<code>Application</code>)
<code>data</code>	<code>DefaultPrograms</code> (in-code program defs, largely vestigial), <code>ExerciseLibrary</code> (muscle-group lookup)
<code>data.db / .db.entity / .db.dao</code>	<code>AppDatabase</code> + 11 entities + 9 DAOs
<code>data.repository</code>	<code>GymRepository</code> — the single data API
<code>data.parser / .parser.smart</code>	Excel import: Default, Bulk, Smart (7-method), Xlsx, Text parsers + <code>ParsedModels</code>

Package	Responsibility
data.ai	ClaudeApiClient, AiFileExtractor, AiModels — AI-assisted import
data.datastore	ThemePreferences (DataStore-backed settings + feature flags)
ui.home / .session / .library / .history / .habits / .create / .settings	Screens + view models per feature area
ui.navigation	AppNavigation (NavHost + bottom-tab pager)
ui.theme / .components	Theme/Color/Typography; GlassCard, ProgressionBadge
widget / receiver	Glance habit widget + boot/midnight receivers + alarm scheduler

6. Data model (Room schema v13)

Room database `gym_tracker.db`, version **13**, `exportSchema = false`, with **12 hand-written migrations (1 -> 13)** and **no destructive-migration fallback** (a missing migration would crash, preserving user data integrity). Eleven entities:

Entity (table)	Key fields	Notes
Program (programs)	name, totalWeeks, category, isActive, isUserCreated, isPaused, isFlexible, trackRpe, trackOneRm, currentPhase, coverImage	One row per program; exactly one isActive at a time
ProgramPhase (program_phases)	programId(FK), phaseNumber, name, durationWeeks, focus	Only present when a program has >1 phase
Session (sessions)	programId(FK), weekNumber, dayNumber, name, muscleGroups, phaseNumber	One per (week, day); weeks are global/continuous across phases
Exercise (exercises)	sessionId(FK), name, sets, repsTarget, notes, orderIndex, rpeTarget, pct1rmTarget, supersetGroupId, unilateralMode	The planned movement; position = (dayNumber, orderIndex)
WorkoutLog (workout_logs)	exerciseId(FK), sessionId(FK), weekNumber, completedAt, progressionChoice	One per exercise per logged week
SetLog (set_logs)	workoutLogId(FK), setNumber, weightKg, reps, rpe, pct1rm, weightRightKg, repsRight	Actual logged set values (per-side fields for unilateral)
SessionCompletion (session_completions)	sessionId(FK), weekNumber, completedAt	Marks a day done; drives Home/History
SessionTemplate (session_templates)	name, muscleGroups, notes, source	Reusable session in the Session Library
SessionTemplateExercise (session_template_exercises)	templateId(FK), name, sets, repsTarget, notes, orderIndex, rpeTarget, pct1rmTarget	Exercises within a library session
Habit (habits)	name, emoji, createdAt	Daily habit
HabitCompletion (habit_completions)	habitId, completedDate 'YYYY-MM-DD' (unique idx)	One per habit per day

Relationships & cascade

- Program 1—N Session 1—N Exercise 1—N WorkoutLog 1—N SetLog — all with `ON DELETE CASCADE`.
- Program 1—N ProgramPhase; Session 1—N SessionCompletion; SessionTemplate 1—N SessionTemplateExercise (cascade).
- **Cross-week carry-forward is positional:** prior-week prefill is looked up by (programId, dayNumber, orderIndex) rather than by exerciseId, so it survives exercise edits/reorders across weeks.

Migration history (additive)

v1->2 coverImage; 2->3 habits + habit_completions; 3->4 isFlexible; 4->5 category; 5->6 isActive/isUserCreated; 6->7 session-name cleanup; 7->8 trackRpe/trackOneRm + set rpe/pct1rm; 8->9 exercise rpeTarget/pct1rmTarget; 9->10 isPaused; 10->11 supersetGroupId; 11->12 unilateralMode + per-side set columns; 12->13 phases (program_phases, phaseNumber, currentPhase) + session library tables.

7. Repository & DAO layer

GymRepository wraps nine DAOs and exposes the full data API. Representative operations:

Group	Operations
Program lifecycle	importProgram / addToLibrary, activateProgram, startProgramFromLibrary, pause / resume / restartPausedProgram, cancelProgram, deleteProgram, restartProgram, restartFromWeek2, startProgramAtPhase, setCurrentPhase
Logging	getOrCreateWorkoutLog, upsertSetLog, deleteSetLog, getSetLogsForWorkoutLog, getSessionSetLogs, updateProgressionChoice
Carry-forward	getPrevWeekSetLogsByPosition, getPrevWeekProgressionByPosition (positional, cross-week)
Day state	completeSession, isSessionCompleted, resetSessionDay (wipe a day's sets/logs/completion)
Exercise editing	add/delete/rename/reorder exercise, updateExerciseSets, link/unlink superset, setUnilateralMode, createExerciseAndPropagate (to future weeks)
Analysis	analyseProgram (sets/week per muscle via ExerciseLibrary), getAllWeekSetLogs (progression charts)
Session library	librarySessions, save/update/deleteSessionTemplate, syncLibraryFromPrograms
Maintenance	clearAllData

8. Navigation map

Single NavHost, start destination main. The main route hosts a 5-tab HorizontalPager (no per-tab back-stack); full-screen flows are separate routes.

Route	Screen
main	MainPagerScreen -> tabs: Home · Library · Habits · History · Settings
session/{sessionId}/{weekNumber}	SessionScreen — live set-logging view
session_info/{sessionId}/{weekNumber}	SessionInfoScreen — read-only view + share (NEW)
create_program	CreateProgramScreen — multi-step builder
session_library	SessionLibraryScreen — reusable session templates

9. Feature catalogue

9.1 Home (active program)

- Banner/header with program name, current week, and completion progress (optional full-width image header via a feature flag).
- Week sections, each a grid of **day cards** showing day number, name, muscle groups, and Done / Next-up state.
- Per day card: tap to open the logging view; **info icon** to open the read-only session view without starting it (NEW).
- **Phase selector** appears only for multi-phase programs; it filters the week list to the selected phase's range.

- Program-level actions: Restart from Week 1, Restart using Week-2 weights as the new baseline, Cancel program; a completion dialog when every session is done.
- Entry points for importing/choosing programs when none is active.

9.2 Session logging (SessionScreen)

- Horizontal pager of exercises (supersets render as a combined page); progress bar by exercises logged.
- Per set: weight x reps, plus optional **RPE** and **%1RM** columns (shown when the program tracks them), and **unilateral** modes (single weight + L/R reps, or full L/R weight+reps).
- **Carry-forward prefill**: last week's values appear in italic/accent as editable defaults; first-set weight auto-fills later sets until edited (toggleable).
- Add/delete sets (swipe to delete); add/rename/reorder/delete exercises; link supersets; per-exercise free-text notes (kept by exercise name).
- Optional rest timer / stopwatch with sound, and a 1RM (Epley) calculator.
- Per-exercise 'how to progress next week' picker; **Complete Session** marks the day done.
- **Reset day** (NEW): overflow menu -> wipes this day/week's logged sets and completion, reverting to carried-over prefill and unmarking it.

9.3 Session info & sharing (NEW)

- Read-only plan view of a session: each exercise with target sets x reps, RPE/%1RM targets, and notes — reachable without starting/logging.
- **Share as plain text** (formatted summary via the Android share sheet) and **Share as image** (a branded PNG rendered on an Android Canvas, shared via FileProvider).

9.4 Library, analysis & program management

- Browse the bundled default-program catalogue (grouped by category, searchable, filter by days/week) and a separate Browse-Defaults sheet.
- Your saved programs, favourites, paused programs; activate / start / restart-paused.
- **Program analysis dashboard**: weekly set volume per muscle group (primary 1.0x, secondary 0.5x via ExerciseLibrary), push/pull ratio, weakest/under-worked muscles vs MEV, balance verdict, and readable insight strings; per-exercise weekly progression snapshots.

9.5 Session Library, Program Builder, History & Habits

- **Session Library**: create/edit/delete reusable sessions; 'import days from my programs' de-dupes program days into templates; load a template into a builder day.
- **Create Program**: multi-step builder (type, name, days/week, weeks, per-day exercises, RPE/%1RM tracking, cover image) producing a user program.
- **History**: list of completed sessions with computed total training volume (kg) per session.
- **Habits**: daily habit list with emoji, toggle-complete-today, streaks; a Glance **home-screen widget** with midnight reset.

9.6 Settings

- Dark mode; accent colour via hue + saturation sliders (default hue 203, sat 0.72) with reset.
- Feature toggles (see §12); timer-sound selection; Claude API key entry; Clear-all-data.

10. Import & parsing subsystem

Importing converts an external source into the normalized in-memory model (ParsedProgram -> ParsedPhase / ParsedSession -> ParsedExercise), which the repository persists. Four ingestion paths exist:

Path	Source	Parser / entry
Smart Excel	Any .xlsx (coach sheets, custom templates, official template)	SmartXlsxParser — 7-method auto-detection; user reviews a preview before import
Default catalogue	Bundled assets/default_programs.xlsx	DefaultProgramXlsxParser — one sheet per program + a Program Index sheet
Bulk / phased	assets/bulk_template.xlsx-shaped workbook	BulkProgramXlsxParser — flat header-driven sheet, many programs, multi-phase; repeats one representative week per phase
AI (any file)	PDF / docs / text via the file extractor	AIFileExtractor (PdfBox etc.) -> ClaudeApiClient -> review screen

A downloadable `template.xlsx` and `bulk_template.xlsx` ship as assets. The official single-program template has Program Info + per-day sheets with columns for Exercise, Sets, Reps, Weight, %1RM, RPE, Rest, Superset, Notes.

11. Default program catalogue (asset-driven)

The browsable catalogue is generated **at runtime** from the bundled spreadsheet `app/src/main/assets/default_programs.xlsx` (parsed by `DefaultProgramXlsxParser`) — **not** from the Kotlin `ALL_DEFAULT_PROGRAMS` list, which is dead/vestigial.

- One worksheet per program named `N-Title`, plus a `Program Index` sheet (Goal drives category; Intensity drives RPE/%1RM tracking).
- Currently **61 programs** across Strength, Hypertrophy, Powerlifting, Powerbuilding, GPP, and Conditioning.
- Each program is a single representative week (the parser does not auto-repeat weeks); the Index 'Weeks' column is informational.
- **Constraint:** the index parser stops after 60 entries — program #61+ falls back to its own sheet metadata for category/tracking.

12. Configuration, feature flags & secrets

DataStore (ThemePreferences) — theming & feature toggles

Key	Type	Default
<code>dark_mode</code>	Bool	false
<code>accent_hue / accent_sat</code>	Float	203.0 / 0.72
<code>feat_session_timer</code>	Bool	true
<code>feat_weight_autofill</code>	Bool	true
<code>feat_add_exercise_mid_session</code>	Bool	true
<code>feat_progression_picker</code>	Bool	true
<code>feat_program_images</code>	Bool	true
<code>feat_1rm_calculator</code>	Bool	true
<code>timer_sound</code>	String	"ding" (also "ronnie", "none")

SharedPreferences

- `gym_prefs`: `claude_api_key` (AI import), favourites (program names).
- `exercise_notes`: per-exercise user notes, keyed by exercise name (global across programs).

AI integration configuration

Setting	Value
Endpoint	https://api.anthropic.com/v1/messages (OkHttp)
Model	claude-haiku-4-5-20251001
max_tokens	4096
anthropic-version	2023-06-01
Auth	User-supplied key in SharedPreferences (x-api-key header)

Secret handling: the Claude API key is stored in plaintext SharedPreferences and sent directly to Anthropic from the device. There is no app backend and no secret beyond this user-provided key.

13. Sharing & FileProvider

- A FileProvider is declared with authority `/${applicationId}.fileprovider` and paths in `res/xml/file_paths.xml` (external-path, files-path, and a cache-path added for shared images).
- Used by: Excel template downloads (via the system document-create contract) and the new session image/text share (image PNG written to cache, shared with a content:// URI and a temporary read grant).
- Image rendering is done with `android.graphics.Canvas` because the project's Compose version (1.6.8) predates the `graphicsLayer` capture API.

14. Background work, receivers & home-screen widget

- **Habit widget** (Glance): `HabitWidget` / `HabitWidgetProvider` / `HabitWidgetReceiver`, with `ToggleHabitCallback`; companion dialog activities `RenameHabitActivity` and `SetStreakActivity`.
- **Midnight reset:** `MidnightAlarmScheduler` sets an exact alarm; `MidnightResetReceiver` refreshes the widget at day rollover; `BootReceiver` re-schedules on boot / app update.
- Permissions declared: `INTERNET`, `RECEIVE_BOOT_COMPLETED`, `SCHEDULE_EXACT_ALARM`, and legacy/scoped media-read permissions for older SDKs.

15. Data & security model (access control / ABAC)

The app is **single-user and local**. There is intentionally **no authentication, no authorization layer, and no role/attribute-based access control** — every feature is available to whoever holds the unlocked device, and all data is the one user's own.

Concern	Current state
Identity / accounts	None. No login, no user records, no multi-user separation.
Authorization	None. No roles, permissions, or attribute checks gating any screen or operation.
Data at rest	Room SQLite DB + DataStore + SharedPreferences in app-private storage, unencrypted . <code>android:allowBackup=true</code> (eligible for system/cloud backup).
Data in transit	Only the optional Claude API call (HTTPS, user's own key). No other network egress; no telemetry/analytics.
Secrets	User's Claude API key in plaintext SharedPreferences.

Roadmap implication: any feature involving accounts, cloud sync, sharing beyond the device, coach/athlete relationships, or multi-profile support would require introducing an identity + access-control layer (and likely at-rest encryption) that does not exist today. Treat that as a foundational workstream, not an add-on.

16. Build, signing & release artifacts

- Build via Gradle wrapper: set JAVA_HOME to Android Studio's JBR, then :app:assembleDebug. Room codegen via KSP.
- **Signing:** a single debug signing config (the standard debug keystore) is applied to **both** debug and release builds.
- **Release type:** isMinifyEnabled = false — no R8/shrinking/obfuscation despite proguard files being referenced.
- No CI/CD; APKs are produced locally and kept under releases/. Latest is a versioned debug APK; a pre-change backup is also stored there.

Implication: the current release output is **not Play-Store-ready** (debug-signed, unminified). Production release would need a real keystore, R8 configuration, and likely an app-bundle pipeline.

17. Quality: tests & static analysis

- **Automated tests:** none — there are no test/ or androidTest/ sources. Verification today is build success + manual checks.
- **Lint:** last run reported 3 errors and 66 warnings. The 3 errors are SuspiciousIndentation false positives caused by the codebase's flat 2-space style (verified harmless). Warnings are routine (~39 outdated-dependency notices, unused resources, hardcoded strings, the compileSdk-36 advisory).

18. Known constraints, tech debt & risks

- **No test suite** — highest-leverage gap for safe iteration; refactors and new features have no regression net.
- **Release not shippable as-is** — debug-signed and unminified (see §16).
- **Single module, no DI** — fine now, but will strain as features grow; the repository is becoming a large surface.
- **Dead code:** ALL_DEFAULT_PROGRAMS in DefaultPrograms.kt has no consumers (the live catalogue is the xlsx asset).
- **Catalogue parser cap** at 60 indexed programs (§11).
- **exportSchema = false** — no Room schema JSON history (harder to diff/migrate confidently).
- **Exercise notes are global by name** in SharedPreferences (not per-program/per-exercise-instance).
- **Toolchain drift:** AGP 8.4.2 vs compileSdk 36; Compose 1.6.8 lacks newer capture/APIs (worked around for image share).
- **Unencrypted local data + allowBackup=true** — acceptable for a personal tracker, a gap the moment data becomes sensitive or shared.

19. Extension points (where new features plug in)

The seams below are where most new capability naturally attaches. Anchor roadmap items to these to keep changes localized.

Seam	What to do here
GymRepository	Add a new cross-table operation as a suspend function; it is the single data API every view model already uses.
Room: entity + migration	Add/extend an entity, bump the DB version, and add a Migration(n, n+1) following the established additive pattern.
ParsedProgram / ParsedSession / ParsedExercise	Normalize any new import source into these models and reuse importProgram — no persistence code needed.
AppNavigation routes	Add a composable route (the session_info route is a recent worked example) and link it from the relevant screen.

Seam	What to do here
ThemePreferences (DataStore)	Add a new feature flag / setting key + default, and a Settings toggle.
ExerciseLibrary	Extend muscle-group mappings to improve the analysis dashboard's coverage.
assets/default_programs.xlsx	Add programs to the shipped catalogue by appending sheets + an Index row (no code change).
ClaudeApiClient	Adjust model/params or add new AI-assisted flows; the OkHttp call is isolated here.

Foundational workstreams a roadmap will likely need

- **Identity & access control** if accounts / sync / sharing / coach-athlete features appear (§15).
- **Test harness** (unit tests for the repository/parsers; instrumented tests for key flows) before large changes.
- **Release engineering** (real keystore, R8, app bundle, CI) before public distribution.
- **Backend / sync layer** if data must leave a single device — none exists today.

20. Recent changes (this work cycle)

- Added 15 programs to the shipped catalogue across two batches (now 61 total) by appending sheets to `default_programs.xlsx`.
- **Reset day:** new DAO deletes + `GymRepository.resetSessionDay` + `SessionViewModel.resetDay`, surfaced via a `SessionScreen` overflow menu with confirm dialog.
- **Session info & share:** new read-only `SessionInfoScreen` / `SessionInfoViewModel` (route `session_info`), an info button on Home day cards, and share-as-image (Canvas PNG via `FileProvider`) / share-as-text.
- All changes compile (`assembleDebug` succeeds); the rebuilt APK was verified to bundle the updated catalogue.

21. Appendix — key source file index

Concern	File(s)
Entry / app	<code>MainActivity.kt</code> , <code>GymApp.kt</code> , <code>ui/navigation/AppNavigation.kt</code>
Database	<code>data/db/AppDatabase.kt</code> , <code>data/db/entity/*.kt</code> , <code>data/db/dao/*.kt</code>
Repository	<code>data/repository/GymRepository.kt</code>
Home	<code>ui/home/HomeScreen.kt</code> , <code>ui/home/HomeViewModel.kt</code>
Logging	<code>ui/session/SessionScreen.kt</code> , <code>ui/session/SessionViewModel.kt</code>
Session info/share	<code>ui/session/SessionInfoScreen.kt</code>
Library/analysis	<code>ui/library/LibraryScreen.kt</code> , <code>ui/library/LibraryViewModel.kt</code>
Session library	<code>ui/library/SessionLibraryScreen.kt</code> + <code>ViewModel</code>
Builder	<code>ui/create/CreateProgramScreen.kt</code> + <code>ViewModel</code>
History / Habits	<code>ui/history/*.kt</code> , <code>ui/habits/*.kt</code>
Settings/theme	<code>ui/settings/*.kt</code> , <code>data/datastore/ThemePreferences.kt</code> , <code>ui/theme/*.kt</code>
Import/parse	<code>data/parser/*.kt</code> , <code>data/parser/smart/*.kt</code> , <code>data/parser/ParsedModels.kt</code>
AI	<code>data/ai/ClaudeApiClient.kt</code> , <code>AiFileExtractor.kt</code> , <code>AiModels.kt</code>
Widget/receivers	<code>widget/*.kt</code> , <code>receiver/*.kt</code>
Assets	<code>assets/default_programs.xlsx</code> , <code>template.xlsx</code> , <code>bulk_template.xlsx</code>

End of document — generated from a direct read of the codebase as it stands on 19 June 2026.